Physical Intelligence $\pi_{0.5}$ — Unified Engine v1

Apex Compute

1 The $\pi_{0.5}$ Model

$\pi_{0.5}$ is a vision–language–action (VLA) policy of roughly three billion parameters that maps camera observations and a language instruction directly to robot actions. Each inference emits a 10×7 action chunk: ten future control steps, each a 7-dimensional command giving the robot's degrees of freedom (arm motion plus gripper). The computation splits into three stages run in sequence—a vision encoder, a language prefix, and a flow-matching action expert.

1.1 Vision encoder

Each image is encoded independently by a SigLIP ViT; $\pi_{0.5}$ takes up to three slots (scene, wrist, one padded). Every 224×224 image becomes 256 patch tokens of width 1152, run through L_v=27 bidirectional layers (16 heads, head-dim 72, MLP width 4304). Re-run per image and never cached, it costs ≈656 GFLOP.

1.2 Language prefix

The patch tokens are projected to d=2048, concatenated with the instruction tokens, and processed once by a Gemma-2B backbone: L_p=18 layers of grouped-query attention (8 query, 1 KV head, head-dim 256), RMSNorm, RoPE, and a gated SiLU MLP (I=16384). At prefill length M_p=576 one layer costs

$$2M_p d\, d_q + 4M_p d\, d_{kv} + 2M_p d_q d + 4M_p^2 d_q + 6M_p d\, I, \tag{1}$$

(d_q=2048, d_{kv}=256), dominated by the MLP; over 18 layers the prefix is ≈2332 GFLOP (76% of the model). Each layer's K/V is cached and reused by every denoise step.

1.3 Action expert

A second, smaller Gemma (∼300 M; H=1024, MLP 4096, 18 layers) turns noise into the action chunk by flow matching: from Gaussian noise it integrates a learned velocity field v_θ over N=10 Euler steps ($\Delta t = -0.1$),

$$x_{t+\Delta t} = x_t + \Delta t\, v_\theta(x_t,\, t,\, \mathrm{KV}_{\mathrm{prefix}})\,, \tag{2}$$

each step attending over the cached prefix K/V and reading out actions through a (1024→32) head. At only ≈71 GFLOP (2.3%) it is arithmetically small but the sole sequential stage, and its per-step error accumulates through the integrator.

Table 1: Per-stage parameter and FLOP budget (per inference). *Effective* uses true model dims; *FPGA* is the padded shape the transformer engine actually issues. The prefix is already 64-aligned (no padding); the vision tower pads only its attention head (72→128, +2.9%); the action expert pads its query horizon 10→64 and action width 32→64, inflating its work ∼6.5× — so overall the FPGA issues ∼13% more FLOPs than the model nominally needs.

Stage	**Params**	**Eff. GFLOP**	**FPGA GFLOP**
Vision tower (SigLIP, 3 slots)	∼0.41 B	655.9	674.9
Language prefix (Gemma-2B)	∼2.0 B	2332.0	2332.0
Action expert (Gemma-300M)	∼0.31 B	71.0	459.4
Total	**∼2.7 B**	**3058.9**	**3466.3**

2 Porting

The Unified Engine v1 is a soft accelerator IP block whose compute core is the Apex Compute Transformer Engine; it can be deployed on any sufficiently large FPGA and operates on 64-element alignment. In this work a single transformer engine is targeted to a Xilinx Kintex-7 with 4 GB of attached DRAM. These two constraints—64-alignment and the 4 GB DRAM budget—drive the porting transformations below.

IF4 weights / DRAM budget. The ∼13 GB bf16 checkpoint does not fit, and writes at/above the 4 GB boundary fail on this board. All matmul weights use 4-bit block quantization (block 64, weight-only): every 64-element block stores 64 INT4/FP4 codes plus *one* bf16 scale, so a quantized matrix costs 4.25 bits/weight ($4 + 16/64$, i.e. 3.76× smaller than bf16 rather than 4×), while norm gains and biases stay bf16 outright. That shrinks the three stacks to ∼1.56 GB. Weights are dequantized to bf16 immediately before each multiply–accumulate, so IF4 is a *footprint enabler, not a speedup* — compute is bf16 throughout.

Vision head-dim pad. Head-dim 72 is not 64-/128-aligned, so the attention matmuls are padded 72→128 and contracted back via a precomputed 128×72 selection matrix; the $Q/K/V/O$ projections stay at 72.

Prefix masking. The masked image slot is dropped, compressing prefill 832→576 (bit-identical, ∼24% faster). Rotary positions use cumulative-sum-of-mask indices, not `arange`, so dropping a slot does not shift phase. The non-causal/causal mixed attention mask is supplied as a static bias tensor in DRAM.

Denoise loop. All 10×18 steps are unrolled into one program issued with a single execute call; AdaRMSNorm conditioning is a matmul to $3H$ split into scale/shift/gate. Every strided SRAM→DRAM copy needs an explicit per-index destination offset — a missing one yields finite-but-scrambled output (not a NaN), and fixing these recovered the expert from −10 to ∼29 dB.

3 Quantization

3.1 The Quantization Floor

IF4 is the accelerator's 4-bit weight format: a per-64-block hybrid of signed INT4 (uniform grid $[-8, 7]$) and FP4 E2M1 (non-uniform codebook $\{0, \pm 0.5, \pm 1, \pm 1.5, \pm 2, \pm 3, \pm 4, \pm 6\}$, finer near zero). At pack time each block is quantized *both* ways and the lower-MSE variant kept; the choice rides in the *sign* of the block's scale (neg → INT4, pos → FP4), so the on-chip dequantizer picks the

decode path from one bit, inline before the MAC—no metadata. INT4 suits dense blocks, FP4 the peaky/outlier ones. (Results below use the INT4 variant, `q4_64`.)

We measure the accuracy cost against a bf16 reference (identical torch run, same inputs/noise, diffed per stage; Table 2, SNR=$20\log_{10}(\|b\|/\|a-b\|)$, masked rows excluded, sample 0)—the theoretical quantization floor, isolated from any hardware effect. Vision is nearly untouched (25.4 dB); the prefix caches degrade with depth, V worse than K (10.7 vs. 16.0 dB mean); and denoise carries the largest loss as per-step error accumulates through Eq. (2)—x_t walks $45 \to 16.3$ dB (Table 3), concentrating in low-magnitude action dims (dim 1, 1.5 dB) while task-critical dims hold (dim 0 24.9, gripper 40.7 dB). SNR is a diagnostic; the acceptance gate is the closed-loop LIBERO agreement (Sec. 3.2).

Table 2: IF4 vs. bf16 per section (torch reference, sample 0).

Model section	**SNR (dB)**	**Range (dB)**
Vision encoder (3 cameras)	25.37	24.3–26.6
Prefix K-cache (18 layers)	15.96	12.5–19.6
Prefix V-cache (18 layers)	10.65	7.0–15.4
Denoise velocity v_t (10 steps)	23.49	11.5–26.2
End-to-end action (10×7)	**16.31**	MSE 4.05×10^{-3}

Table 3: Final-action detail: per-dimension SNR (left) and error accumulation of the running state x_t across Euler steps (right). Sample 0.

Action dim	**SNR (dB)**	**Euler step**	x_t **SNR (dB)**
dim 0 (primary)	24.91	step 0	44.95
dim 1	1.50	step 3	30.13
dim 2	10.08	step 5	24.15
dim 3	6.73	step 7	20.53
dim 4	4.65	step 8	19.26
dim 5	17.61	step 9	16.31
dim 6 (gripper)	40.73	(final)	16.31

3.2 LIBERO

LIBERO is a standard tabletop manipulation benchmark for evaluating language-conditioned robot policies: a suite of pick-and-place and articulated tasks in simulation, scored by closed-loop task success over many episodes. Its observation uses **two** camera views—a fixed scene camera and a wrist-mounted camera. $\pi_{0.5}$, however, exposes **three** image slots, and we process all three: the two real LIBERO cameras plus a third all-zero (attention-masked) slot. Encoding the empty slot is intentional—other embodiments and benchmarks drive the policy with a genuine third camera for their robot actions, so keeping the full 3-slot path exercised on hardware makes the port drop-in for those settings at no correctness cost (the masked slot contributes nothing to the output).

An episode is many inferences. Each model call in Sec. 1 predicts a 10×7 chunk—ten future control steps. LIBERO runs *closed-loop*: the simulator executes the first k=10 steps of that chunk, then feeds the resulting new observation back to the policy for a fresh chunk, and repeats. So a single task *episode* is a *sequence* of inferences—one every k sim steps—not a one-shot prediction, and the episode runs until the task's goal predicate fires (success) or a per-suite step cap is hit

(failure). An episode therefore takes $\lceil \text{steps}/k \rceil$ inferences: the successful runs in Table 4 are ∼10–16 inferences, the capped failures ∼23–29. This is what makes the closed-loop test meaningful and also much heavier than a single forward pass—every one of those inferences runs the full vision + prefix + 10-step denoise pipeline on hardware.

Per-tensor SNR is only a diagnostic; the real acceptance test is whether the quantized policy *acts* the same across that whole loop. Table 4 is the closed-loop comparison—the FPGA IF4 policy vs. the CUDA bf16 reference over 10 prompts (9 `libero_spatial` + 1 `libero_object`), one episode each, with identical seed and fixed denoise noise so both platforms see byte-identical observations. We score each by task success and by the per-episode step count.

Table 4: LIBERO closed-loop agreement: FPGA (IF4) vs. CUDA (bf16), 10 prompts, 1 episode each, fixed seed/noise. ✓=success, ×=failure; the number is episode length in sim steps. Δ is FPGA − CUDA steps on successes (both failures run to the step cap, so Δ is not meaningful there). The two policies reach the *same* outcome on every prompt.

Prompt	FPGA (IF4)	CUDA (bf16)	Δ steps	Agree
0	✓ 94	✓ 95	−1	✓
1	✓ 160	✓ 126	+34	✓
2	✓ 123	✓ 115	+8	✓
3	✓ 108	✓ 109	−1	✓
4	× 230	× 230	—	✓
5†	× 290	× 290	—	✓
6	✓ 121	✓ 120	+1	✓
7	× 230	× 230	—	✓
8	✓ 107	✓ 106	+1	✓
9	× 230	× 230	—	✓
Rate	**6/10**	**6/10**		**10/10**

†Prompt 5 is a `libero_object` task; the original `libero_spatial` task 5 exercised a denoise-loop compile path that crashed the FPGA bring-up, so it is substituted with the next unused prompt.

Takeaway. IF4 is not just numerically close—it is behaviorally identical. Across all 10 prompts the FPGA IF4 policy and the bf16 reference reach the *same* success/failure outcome (10/10 agreement, 6/6 shared successes), and on the successful episodes their trajectory lengths differ by only a handful of steps (Δ within a few percent, +34 on one longer reach). The 16.3 dB end-to-end SNR floor of Sec. 3 sounds alarming in isolation, but the low-SNR components are small-magnitude action dimensions that do not change what the robot *does*—the task-critical axes and gripper are preserved, and the closed-loop behavior confirms it. Put as a benchmark number: quantization *did not regress task performance at all* — same 6/10 success rate, same outcome on every prompt, successful trajectories within a few control steps — so the ∼8× checkpoint compression is free at the task level on this suite.

4 Throughput: FPGA vs. CUDA

We time each stage on both platforms and report it in the same form: per-section wall-clock and the achieved rate (section FLOPs ÷ time). FPGA times come from the engine's own instrumentation. The GPU baseline is openpi's JAX/XLA `Pi0` — the deployment path, sustaining 130.2 ms/inference (7.68 Hz), batch 1 — broken into the same three stages by jitting each separately and forcing device completion (`block_until_ready`) at the boundaries. Because these stages are dispatched separately

rather than fused into the deployed `lax.while_loop`, their sum (135.4 ms) slightly exceeds the fused end-to-end (130.2 ms); the per-stage split is used for the breakdown, the fused number for the headline.

Table 5: RTX 5070 (bf16, openpi JAX/XLA) per-section timing and achieved throughput. Batch 1, 10 denoise steps, sample 0. Per-stage times are separately-jitted; End-to-end is the fused `policy.infer`.

Model section	**Time (ms)**	**GFLOP/s**
Vision encoder	19.41	33 793
Prefix (Gemma-2B)	79.29	29 411
Action expert (denoise)	36.70	1 935
End-to-end	130.15	23 502
Throughput (inf/s)	7.68	

Table 6: FPGA (IF4) per-section timing and achieved throughput (Unified Engine v1 on Xilinx Kintex-7). Batch 1, 10 denoise steps, sample 0, 3-slot vision, 832-token prefix. GFLOP/s and % peak are on the hardware-issued (padded) FLOPs; theoretical peak is 26.6 GFLOP/s (one Apex Compute Transformer Engine at 208 MHz).

Model section	**Time (s)**	**Share**	**Eff. GFLOP**	**HW GFLOP**	**GFLOP/s (hw)**	**% peak**
Vision encoder	35.4	18.3%	655.9	850.7	24.0	90%
Prefix (Gemma-2B)	137.6	71.1%	3399.8	3399.8	24.7	93%
Action expert (denoise)	20.6	10.6%	74.7	483.5	23.5	88%
End-to-end	**193.6**	100%	**4130.4**	**4734.0**	**24.5**	**92%**
Throughput (inf/s)			0.0052			

Table 7: Silicon (IF4, *estimated*) per-section timing and achieved throughput — projected ASIC implementation of the Unified Engine v1 IP at 1.6 GHz with 100 GB/s DRAM, scaling the Apex Compute Transformer Engine count in two chip variations — **Chip Variation 1** (8 engines, ∼5 W) and **Chip Variation 2** (16 engines, wattage TBD) — 1638.4 / 3276.8 GFLOP/s aggregate peak. Batch 1, 10 denoise steps. Linear 61.5× / 123× compute scaling of the measured FPGA run at its achieved %-of-peak; neither variation is bandwidth-bound (see below).

	Chip Variation 1 (8 engines, ∼5 W)		**Chip Variation 2** (16 engines, wattage TBD)	
Model section	**Time (ms)**	**GFLOP/s**	**Time (ms)**	**GFLOP/s**
Vision encoder	575	1 479	288	2 958
Prefix (Gemma-2B)	2 236	1 520	1 118	3 041
Action expert (denoise)	335	1 444	167	2 889
End-to-end	3 146	1 505	1 573	3 010
Throughput (inf/s)	0.32		0.64	

Why the projection is linear. Going from one Apex Compute Transformer Engine at 208 MHz to 8 engines at 1.6 GHz multiplies compute by 61.5× (26.6 → 1638.4 GFLOP/s peak) but DRAM bandwidth by only 16.7× (∼6 → 100 GB/s), so each section survives as compute-bound only if its arithmetic intensity clears the ASIC ridge point of 1638.4/100 ≈ 16 FLOP/byte. Traffic here is dominated by streaming the IF4 weights (4.25 bits/weight, Sec. 2): the prefix reads

$\sim$1.1 GB once for its 3399.8 GFLOP ($\sim$3200 FLOP/byte), the vision tower $\sim$0.65 GB over three slots ($\sim$1300 FLOP/byte), and the worst case — the action expert, whose $\sim$0.16 GB are re-streamed on *every* denoise step against just $\sim$48 GFLOP of padded work per step — still lands at $\gtrsim$200 FLOP/byte with activation and KV-cache traffic included. All three sections sit an order of magnitude above the ridge (worst-section demand $<$10 GB/s of the 100 available), and doubling to 16 engines (Chip Variation 2) only lifts the ridge to $\approx$33 FLOP/byte — still $\geq$6$\times$ clear. So neither variation is bandwidth-bound, and times extrapolate linearly at the FPGA's measured 88–93% of peak, assuming ideal engine scaling: 3.15 s per inference (0.32 inf/s) for Chip Variation 1 and 1.57 s (0.64 inf/s) for Chip Variation 2 — 61.5$\times$ / 123$\times$ the FPGA, within 24$\times$ / 12$\times$ of the RTX 5070's 7.68 inf/s. Either way the engine executes the whole vision–prefix–denoise pipeline autonomously: host work per action chunk is an embedding gather on the way in and a 10$\times$7 readback on the way out, so the platform's CPU stays free for the robot stack.

Power. The RTX 5070 is a 250 W-TGP board; Chip Variation 1 is specified at $\sim$5 W, and Chip Variation 2's wattage is TBD. Table 8 collects the platform comparison.

Table 8: Platform comparison: measured RTX 5070 and Unified Engine v1 FPGA runs, and the two projected chip variations. Achieved GFLOP/s counts effective FLOPs on the GPU and hardware-issued FLOPs on the Unified Engine platforms (Tables 5–7).

	RTX 5070	**FPGA (Kintex-7)**	**Chip Var. 1**	**Chip Var. 2**
Compute	6144 CUDA cores	1 transformer engine	8 engines	16 engines
Clock	$\sim$2.5 GHz	208 MHz	1.6 GHz	1.6 GHz
Peak GFLOP/s	—	26.6	1 638	3 277
DRAM bandwidth	672 GB/s	$\sim$6 GB/s	100 GB/s	100 GB/s
Power	250 W (TGP)	—	$\sim$5 W	TBD
Achieved GFLOP/s	23 502	24.5	1 505	3 010
Time / inference	0.130 s	193.6 s	3.15 s	1.57 s
Throughput	7.68 inf/s	0.0052 inf/s	0.32 inf/s	0.64 inf/s

Config. FPGA: Unified Engine v1 IP (one Apex Compute Transformer Engine) on Xilinx Kintex-7 at 208 MHz (26.6 GFLOP/s theoretical peak), IF4 weights / bf16 compute, 4 GB DRAM at $\sim$6 GB/s. GPU: RTX 5070 (6144 CUDA cores, 250 W TGP), bf16, 12 GB GDDR7 at 672 GB/s, openpi JAX/XLA. Silicon: Chip Variation 1 — 8 engines at 1.6 GHz, $\sim$5 W; Chip Variation 2 — 16 engines at 1.6 GHz, wattage TBD; both 100 GB/s DRAM. All: batch 1, 10 denoise steps, sample 0.

The entire $\pi_{0.5}$ VLA — vision, Gemma-2B prefix, and the 10-step flow-matching expert — runs on the Unified Engine v1. The recurring theme is the split between where FLOPs concentrate (the compute-bound prefix) and where latency does (the sequential, FLOP-light denoise loop), and how IF4 trades a controlled, section-localized accuracy cost — one with no measured closed-loop task regression (Sec. 3.2) — for fitting a $\sim$13 GB model in a sub-4 GB budget. Tables 1–8 complete the characterization.